Distributed systems

A distributed system is a collection of independent computers that appear to the users of the system as a single coherent system. These computers or nodes work together, communicate over a network, and coordinate their activities to achieve a common goal by sharing resources, data, and tasks.

- A distributed system is not just one computer.
- It is made of many computers (nodes) that work together.
- Each computer has its own memory and processor, so they are independent.
- Example: Think of several laptops or servers connected in a company network.

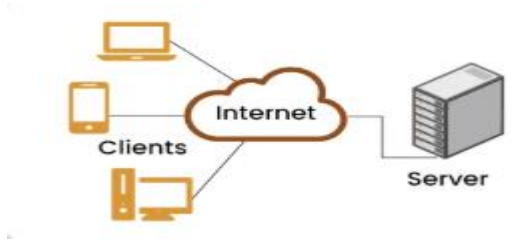
Architectures of Distributed systems

Below are some of the common distributed system architectures:

- **Client-Server Architecture:**

  In this setup, servers provide resources or services, and clients request them. Clients and servers communicate over a network.

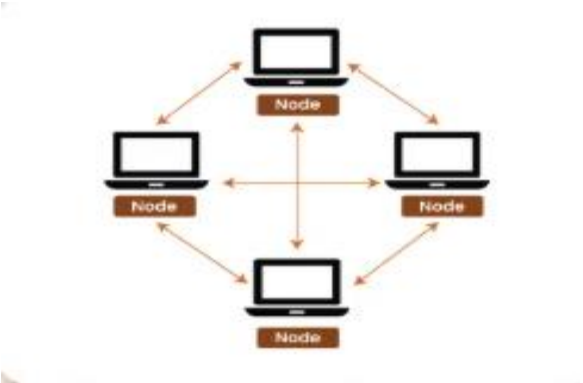
  Examples: Web applications, where browsers (clients) request pages from web servers.



- **Peer-to-Peer (P2P) Architecture:**

Each node, or "peer," in the network acts as both a client and a server, sharing resources directly with each other.

Examples: File-sharing networks like BitTorrent, where files are shared between users without a central server.



Example: Bit Torrent File Sharing

In Bit Torrent, when you download a movie or a file, you are not downloading from one central server.

Instead:

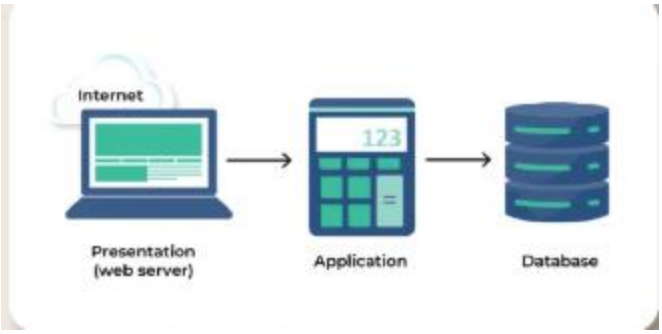
Your computer (a peer) connects directly with many other computers (other peers).

Each peer shares small chunks of the file with others.

At the same time, your computer also uploads chunks to others.

Three-Tier Architecture:

- This model has three layers: presentation (user interface), application (business logic), and data (database). Each layer is separated to allow easier scaling and maintenance.
- **Examples**: Many web applications use this to separate user interfaces, logic processing, and data storage.



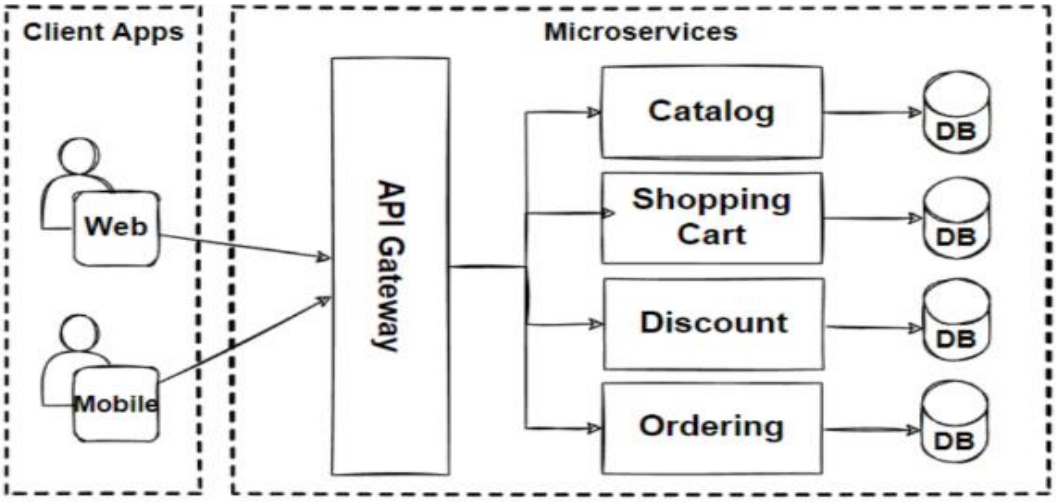
- **Microservices Architecture:**

- The application is split into small, independent services, each handling specific functions. These services communicate over a network, often using REST APIs or messaging.
- **Examples**: Modern web applications like Netflix or Amazon, where different services handle user accounts, orders, and recommendations independently.

Microservices Architecture = A big application is broken into **small independent services**, each doing **one job well**, and they communicate with each other (often through APIs).

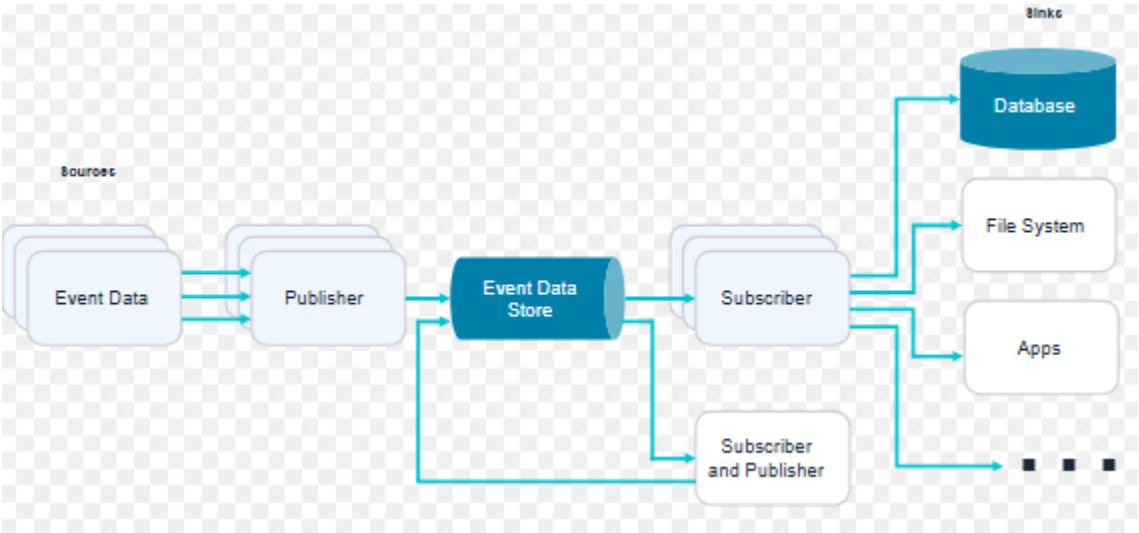
- **Amazon** → product catalog, payment, shipping, recommendation – all separate microservices.

- **Uber** → ride-matching, billing, driver management, GPS tracking, and notifications – each as a separate service.



- **Service-Oriented Architecture (SOA):**
  - Similar to microservices, SOA organizes functions as services. However, SOA typically uses an enterprise service bus (ESB) to manage communication between services.
  - **Examples**: Large enterprise applications in finance or government, where different services handle various aspects of business processes.

- **Event-Driven Architecture:**
  - Components interact by sending and responding to events rather than direct requests. An event triggers specific actions or processes in various parts of the system.
  - **Examples**: Real-time applications like IoT systems, where sensors trigger actions based on detected events.



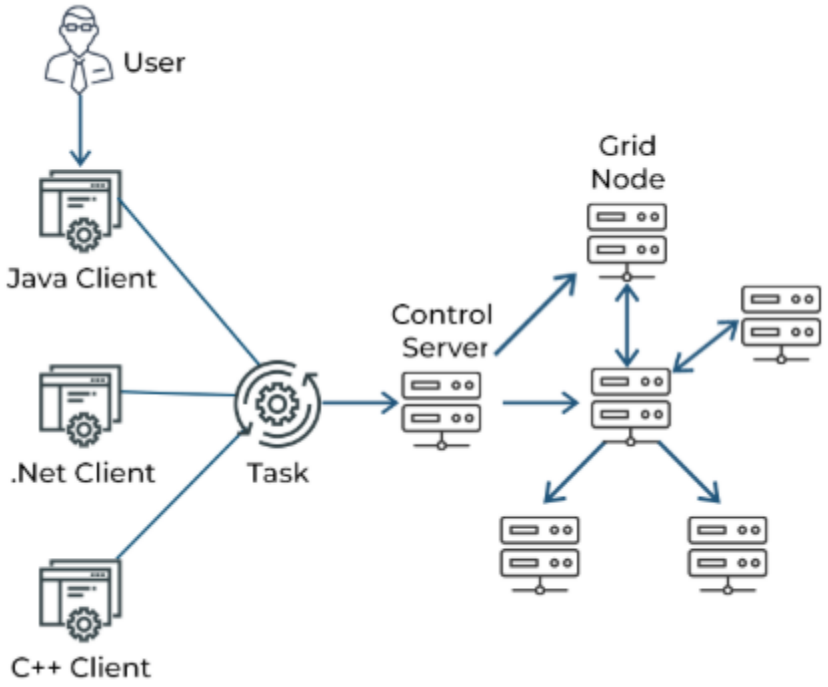
Key Difference Table

Feature	SOA	Microservices
Granularity	Larger services	Smaller, fine-grained services
Communication	Enterprise Service Bus (ESB)	Lightweight protocols (REST, gRPC, MQ)
Database	Often shared	Each service can have its own
Deployment	Services may depend on each other	Each service deploys independently
Technology stack	More uniform	Can be mixed (different languages, DBs)
Focus	Integration of enterprise systems	Building cloud-ready, scalable apps

Grid Computing

Grid computing is a type of distributed computing where computers from different locations (sometimes worldwide) are connected through a network to work together on a common task.

- It uses the idle processing power and storage capacity of many machines.
- The computers are usually loosely coupled (not in the same place).
- The main goal is to solve large, complex problems that a single computer cannot handle easily.



- **User**
  - A user submits a task (like a scientific calculation, data analysis, etc.).
  - The user can use different platforms/programming languages such as Java Client, .Net Client, or C++ Client to send the task.
- **Task Manager**
  - All requests (tasks) go to a central Task Manager (shown as a gear symbol).
  - The task manager prepares the job and sends it to the Control Server.
- **Control Server**
  - This is the central controller of the grid.
  - It decides how to split the big task into smaller subtasks.
  - Then, it assigns these subtasks to different Grid Nodes.

- **Grid Nodes**
  - These are the computers connected in the grid network (can be in different locations).
  - Each node works on a small part of the task.
  - For example, if the task is analyzing climate data, one node may analyze temperature data, another humidity, another rainfall, etc.

- **Execution & Results**
  - After processing, all nodes send their results back to the control server.
  - The control server then combines results and sends the final output to the user.

Cluster Computing

Cluster computing means using a group of computers (nodes) connected through a high-speed local network to work together as if they are one single powerful computer.

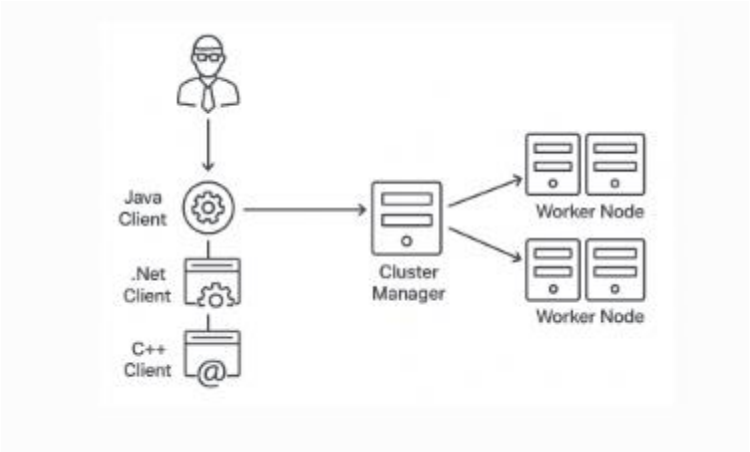
- The nodes usually sit in the same physical location (like a server room or data center).
- They share resources and run tasks in parallel to achieve high performance, fault tolerance, and availability.

How It Works (Flow)

1. **User**
   - A user submits a job (like simulation, data analysis, or running an app).
2. **Cluster Manager (or Master Node)**
   - The master node receives the job.
   - It divides the job into smaller tasks**.**
   - Then, it assigns tasks to worker nodes.
3. **Worker Nodes (Slave Nodes)**
   - Each worker node executes its assigned part of the task.
   - They are tightly connected via a high-speed network (like Ethernet or InfiniBand).
4. **Result Collection**
   - All nodes send their processed results back to the master node.
   - The master node **combines** the results and sends the final output to the user.

Example

- A **weather forecasting cluster**:
  - Cluster of 1000+ computers works together.
  - Each computer calculates weather data for a specific region.
  - Finally, results are merged to predict global weather.



Feature	Cluster Computing	Grid Computing
Definition	Group of computers tightly connected in one location, working like a single system.	Collection of computers (possibly worldwide) loosely connected to share resources.
Location	Nodes are usually in the **same place** (data center, server room).	Nodes can be in **different places**, even different organizations/countries.
Coupling	**Tightly coupled** (high-speed local network).	**Loosely coupled** (Internet or wide-area networks).
Management	Managed by a **cluster manager** (centralized).	Managed by a **control server or middleware** (distributed resource management).
Resource Sharing	Nodes share common resources and storage.	Each node may belong to a different organization and share only some resources.
Scalability	Less scalable (limited to local setup).	Highly scalable (can add computers from anywhere).
Use Case	High Performance Computing (HPC) like weather forecasting, simulations, data analysis.	Large-scale resource sharing like SETI@home, Folding@home, scientific collaboration.
Example	Google Search Engine clusters, University supercomputer.	Worldwide scientific projects like CERN's LHC Computing Grid.

Mobile Computing

Mobile Computing is a type of computing that allows people to access data, applications, and services anytime and anywhere using wireless devices such as smartphones, tablets, laptops, and PDAs.

It combines hardware (devices), software (apps, OS), and communication technologies (Wi-Fi, 4G/5G, Bluetooth, etc.) to provide mobility.

Examples of Mobile Computing Devices:

- Smartphones
- Tablets
- Laptops and ultrabooks
- Wearables (smartwatches, AR/VR headsets)
- IoT devices with mobile connectivity

Advantages

- Easy access to information.
- Improves productivity (work from anywhere).
- Useful for business, education, and entertainment.

Disadvantages

- Security issues (data theft, hacking).
- Battery dependency.
- Network availability problems in remote areas.

Cloud Types (NIST Model)

- According to NIST (National Institute of Standards and Technology), there are four deployment models of cloud:

Public Cloud

A **public cloud** is a type of cloud where services like storage, servers, and applications are provided by a third-party company through the internet. Many users and organizations share the same cloud, but each user's data is kept safe and separate. The best part is that you only pay for what you use, which makes it cheaper than buying your own servers.

Public clouds are easy to scale up or down, meaning you can increase resources when your work grows and reduce them when you don't need as much. Big companies like Amazon (AWS), Microsoft (Azure), and Google (GCP) provide public cloud services.

They are useful for things like hosting websites, storing files, using email services, and running apps. However, since the cloud is shared, it may not be the best choice for organizations that need very high security, like banks or government offices.

Private Cloud

A private cloud is a cloud that is used by only one organization. Unlike the public cloud, it is not shared with other users. It can be managed either by the organization itself or by a third-party company, but all the resources are kept private.

Private clouds give more security and control because the data and applications stay within the organization. This makes them a good choice for banks, government offices, hospitals, or any company that works with sensitive information.

However, building and managing a private cloud can be more expensive, since the organization has to buy and maintain its own hardware or pay for dedicated hosting.

Example:

- A **bank** may use a private cloud to store customer account details and financial transactions safely.
- A **hospital** may use a private cloud to keep patient health records secure.

Hybrid Cloud

A hybrid cloud is a mix of public cloud and private cloud. It allows an organization to use both at the same time. Some data and apps stay in the private cloud (for more security), while others run in the public cloud (for cost savings and flexibility).

Example:

- A retail company can keep customer payment information in a private cloud for safety, but use a public cloud to handle online shopping during big sales .

Community Cloud

A community cloud is a cloud shared by several organizations that have similar needs, like security or legal rules. Instead of each making its own cloud, they share one that suits all of them.

Example:

- A group of **hospitals** may share a **community cloud** to store patient records securely and follow healthcare laws.
- Several **universities** may use a community cloud for research data.

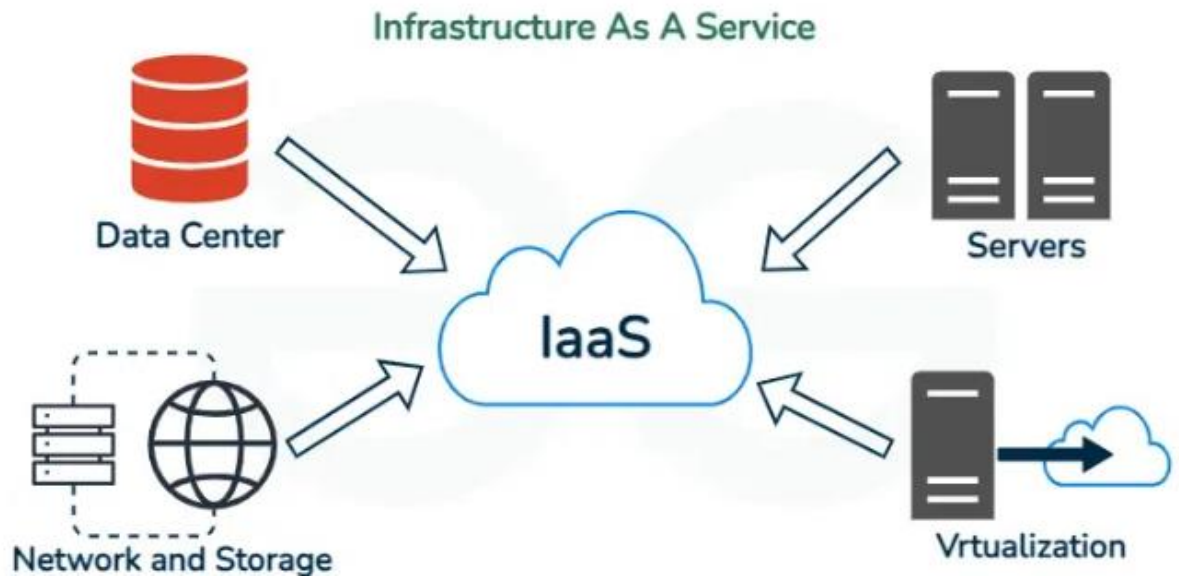
NIST Service Models (Cloud Services)

. Infrastructure as a Service (IaaS)

IaaS gives users **basic IT resources** like virtual machines, storage, and networking over the internet. Instead of buying physical servers, organizations can rent them from cloud providers.

The user has control over the operating system, storage, and apps, but does not worry about the physical hardware.

Example: Amazon EC2, Google Compute Engine, Microsoft Azure Virtual Machines.



Platform as a Service (PaaS)

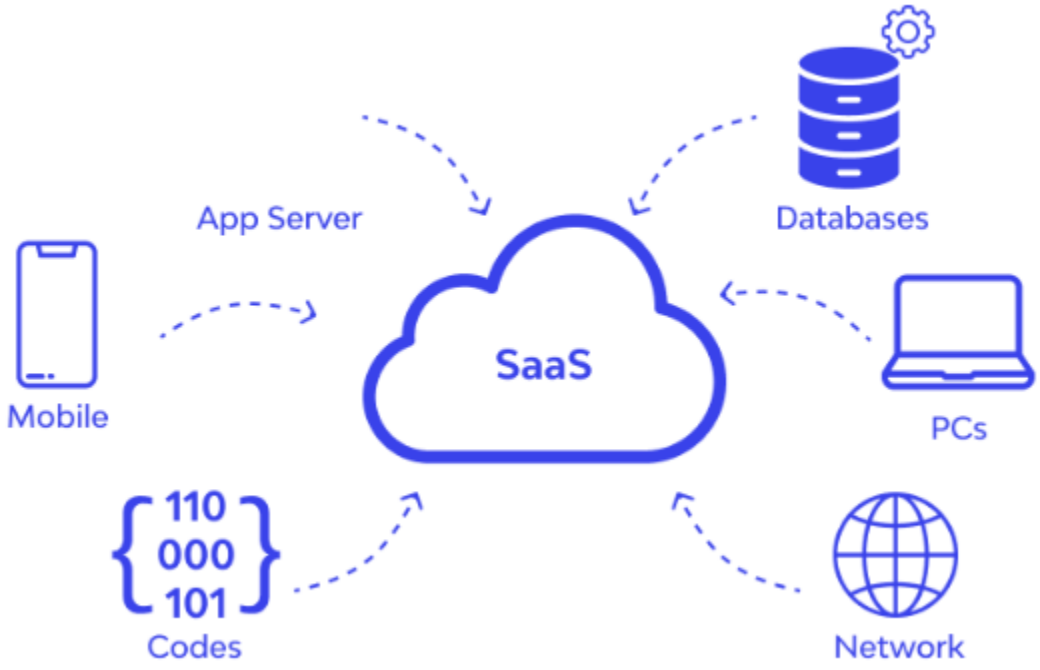
PaaS provides a **ready-made platform** for developers to build, test, and deploy applications. In this model, the user does not manage servers or storage; everything is handled by the provider. The developer can focus only on writing code and creating apps.

PaaS = A platform for developers to create, test, and run apps without managing servers.

Examples: Google App Engine, Heroku, Microsoft Azure App Service, AWS Elastic Beanstalk, IBM Cloud Foundry, Red Hat OpenShift.

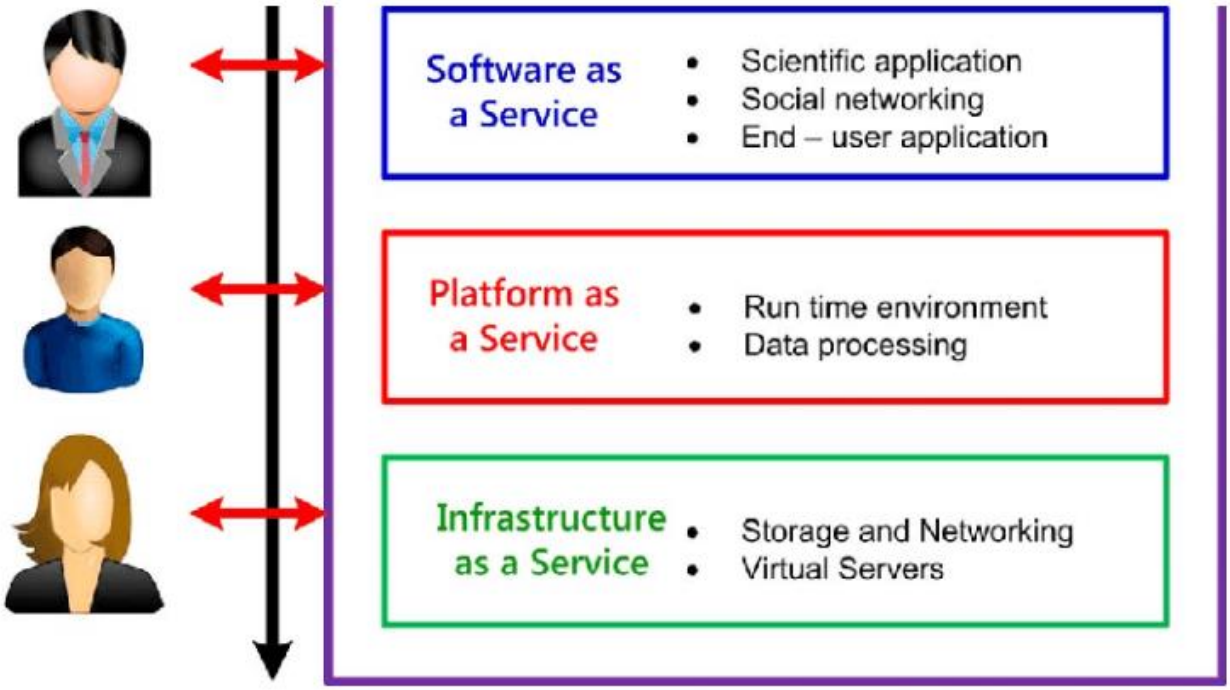
3. Software as a Service (SaaS)

SaaS delivers **ready-to-use software applications** over the internet. Users don't need to install or manage the software on their computers; they can simply access it through a browser or app. The cloud provider takes care of updates, security, and maintenance.



Example: Gmail, Microsoft Office 365, Google Docs, Zoom.

- **IaaS = Renting infrastructure (servers, storage).**
- **PaaS = Ready platform for app development.**
- **SaaS = Ready software to use directly.**



Difference between Cloud Deployment Models:

Feature	Public Cloud	Private Cloud	Hybrid Cloud	Community Cloud
Who uses it?	Open to everyone (general public, companies, individuals)	Only one organization	One organization using both public + private	A group of organizations with common needs
Ownership	Owned by cloud provider (like AWS, Google, Azure)	Owned/managed by one organization or vendor	Mix of public and private ownership	Shared by several organizations

Feature	Public Cloud	Private Cloud	Hybrid Cloud	Community Cloud
Location	Resources are at provider's data centers	Usually on company premises or dedicated host	Mix of on-premises + provider's data centers	Shared data centers for specific community
Cost	Low cost (pay-as-you-go)	High cost (dedicated resources)	Balanced cost (mix of both)	Shared cost among members
Security	Less secure (shared with others)	Very secure (fully controlled)	Secure + flexible (depends on design)	Medium to high security (shared policies)
Scalability	Highly scalable	Limited scalability	Highly scalable (public part)	Moderate scalability
Examples	Gmail, Google Drive, Dropbox	Bank's private cloud, Hospital data cloud	Retailer using AWS + private cloud for payments	Hospitals sharing healthcare cloud, Universities sharing research cloud

Difference Between IaaS, PaaS, and SaaS

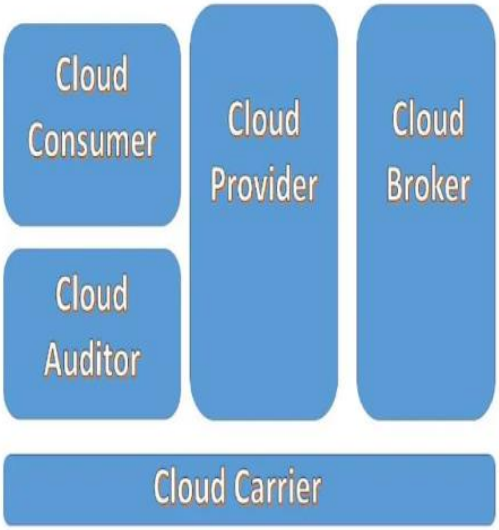
Feature	IaaS (Infrastructure as a Service)	PaaS (Platform as a Service)	SaaS (Software as a Service)
What it provides	Virtual servers, storage, and networks	A ready platform to build and deploy apps	Ready-to-use applications
User controls	User manages OS, apps, data	User manages only the app code and data	User just uses the app, no control over backend
Who uses it?	System admins, IT teams	Developers, programmers	End users (general users, businesses)
Flexibility	Very flexible (full control of infrastructure)	Medium flexibility (control of app only)	Least flexible (just use the app)
Cost	Pay for resources (like renting hardware)	Pay for development platform	Pay subscription or license fee
Examples	AWS EC2, Google Compute Engine, Microsoft Azure VMs	Google App Engine, AWS Elastic Beanstalk, Heroku	Gmail, Zoom, Microsoft Office 365, Google Docs

Cloud Reference Model

The **Cloud Reference Model** is like a **blueprint** that explains how cloud computing works in layers. It shows the **roles**, **components**, and **service models** in the cloud.

Main Layers of the Cloud Reference Model

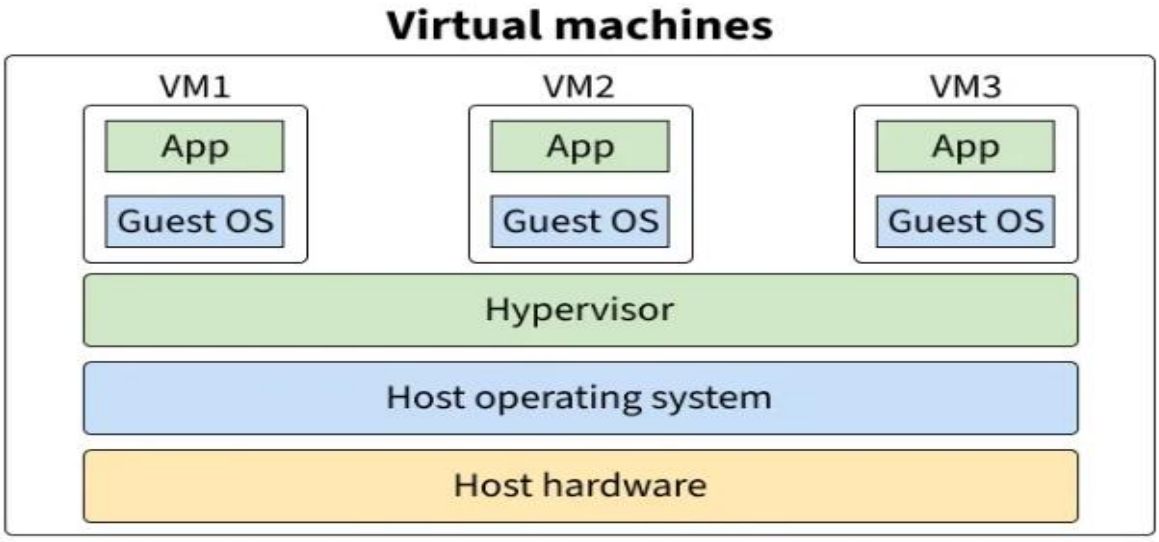
1. **Cloud Consumers (Users)**
   - People or organizations that use the cloud.
   - They access services like storage, apps, or platforms through the internet.
   - Example: A student using Google Drive or a company using AWS servers.
2. **Cloud Services**
   - The main services provided by the cloud:
     - **IaaS** → Virtual machines, storage, networks
     - **PaaS** → Platforms to build and deploy apps
     - **SaaS** → Ready-to-use apps like Gmail, Office 365
3. **Cloud Providers**
   - Companies that deliver the cloud services.
   - They manage data centers, servers, software, and networks.
   - Example: Amazon (AWS), Microsoft (Azure), Google (GCP).
4. **Cloud Infrastructure**
   - The physical resources that make the cloud work:
     - **Servers**
     - **Storage systems**
     - **Networking devices**
     - **Virtualization software**
5. **Cloud Access**
   - The way users connect to cloud services.
   - Usually through the **internet**, web browsers, or mobile apps.
6. **Cloud Security & Management**
   - Includes monitoring, authentication, encryption, backup, and compliance.
   - Makes sure cloud is **safe, reliable, and efficient**.



Virtual Machine (VM)

A **Virtual Machine (VM)** in cloud computing is a software-based computer that runs inside another physical computer. It behaves just like a real computer with its own operating system, memory, and storage, but it is not a separate physical device. Many virtual machines can run on a single physical server, which helps to use hardware resources efficiently.

For example, suppose a cloud provider has one powerful physical server. It can create several virtual machines on that server — one VM running Windows, another running Linux, and another running macOS. Each virtual machine works independently, just like separate computers. This allows users to run different applications, test software, or host websites without needing multiple physical machines.



Cloud Virtualization

Virtualization is the process of creating virtual versions of physical resources such as servers, storage, networks, or operating systems in a cloud environment. Instead of using one physical computer for a single task, virtualization allows many **virtual machines (VMs)** to run on the same physical hardware with the help of a **hypervisor**. Each VM works like a separate computer, having its own operating system and applications. This helps in better use of hardware, reduces cost, and increases flexibility in managing cloud resources.

Types of virtualizations

1. **Server Virtualization:**
   It divides a physical server into multiple virtual servers. Each virtual server can run its own operating system and applications.
   Example: One physical server can host several virtual servers running different apps.

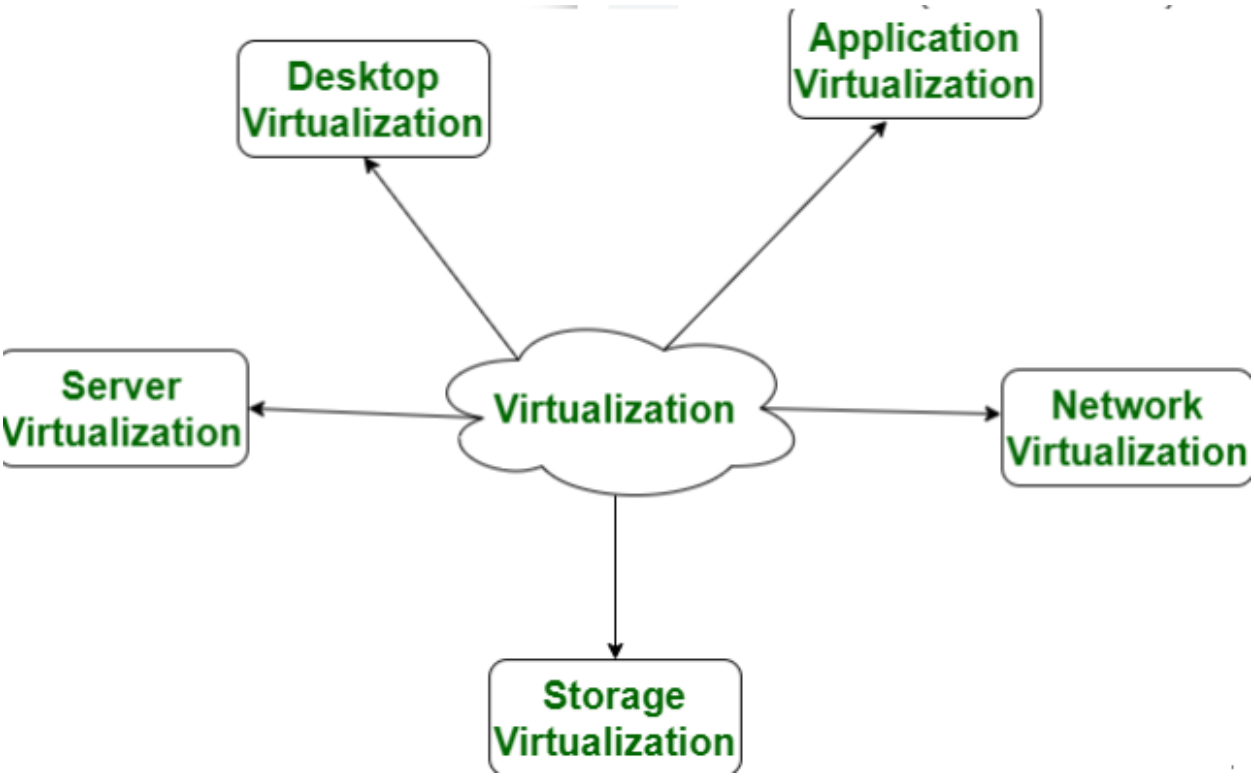
2. **Storage Virtualization:**
   It combines multiple physical storage devices into a single virtual storage unit that users can access easily.
   Example: Several hard drives are shown as one large storage system to users.

3. **Network Virtualization:**
   It creates virtual versions of network resources such as switches, routers, and firewalls to manage traffic efficiently.
   Example: Virtual LANs (VLANs) that separate network traffic within the same physical network.

4. **Desktop Virtualization:**
   It allows users to access their desktop environment from anywhere using the internet. The desktop actually runs on a remote server, not on the local device.
   Example: Using a virtual desktop in the cloud to access files and software from home or office.

5. **Application Virtualization:**
   It allows applications to run on devices without being installed locally.
   Example: Running Microsoft Word or Excel through a virtual app service without installing it on your computer.

6. **Operating System Virtualization (or OS-level Virtualization):**
   It allows multiple isolated user spaces (containers) to run on a single OS.
   Example: Docker containers running multiple applications on one system independently.



Hypervisor

A **hypervisor** is a special software (sometimes called **Virtual Machine Manager – VMM**) used in **cloud hosting**.

- It divides and shares the hardware (CPU, memory, storage) among many virtual machines (VMs).
- It allows many operating systems (Windows, Linux, etc.) to run at the same time on one physical computer.
- It provides partitioning, isolation, and abstraction, so each VM works like a separate computer.

Example: VMware ESXi, VirtualBox.

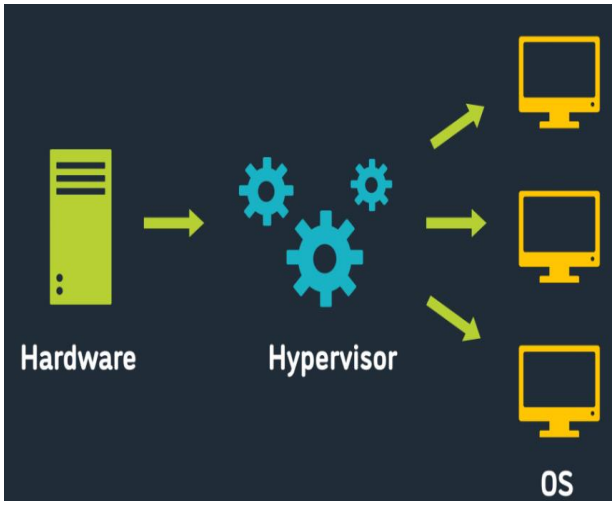
Types of Hypervisor -

A) Type 1 Hypervisor: It runs directly on the hardware (no need for any operating system under it). It is also known as a "Native Hypervisor" or "Bare metal hypervisor". or Bare-metal Hypervisor.

- It can directly use the computer's hardware (CPU, memory, storage).
- Common examples are: VMware ESXi, Citrix XenServer, and Microsoft Hyper-V.

B) **Type 2 Hypervisor :** It runs **on top of a normal operating system** (like Windows, Linux, or macOS).

- It is also called a Hosted Hypervisor.
- It works like a software application that is installed on the host OS.
- It does not talk directly to the hardware; instead, it uses the host OS to access hardware.
- Examples: VMware Player, Oracle Virtual Box, Parallels Desktop.
- These are often used on PCs and laptops by engineers, developers, and security analysts.



CAP theorem:

The CAP Theorem (Consistency, Availability, Partition tolerance) states that in a distributed cloud database system, it is impossible to guarantee all three properties at the same time. A system can only provide two out of three:

- Consistency (C): Every read gives the most recent write (all nodes show the same data).
- Availability (A): Every request gets a response, even if some nodes fail.
- Partition Tolerance (P): The system continues to work even if network failures split communication between nodes.

In cloud databases:

- Consistency + Availability, you must sacrifice Partition Tolerance (not practical in large cloud systems).
- Consistency + Partition Tolerance, you may lose some Availability.
- Availability + Partition Tolerance, you may lose strict Consistency.

Example:

- CP systems: HBase, MongoDB (strong consistency, less availability).
- AP systems: Cassandra, DynamoDB (high availability, eventual consistency).

Load Balancing in Virtualized Environments

- In a virtualized environment, many virtual machines (VMs) run on top of physical servers using a hypervisor.
- Sometimes, one server may get overloaded (too many VMs, too many user requests), while another server is sitting underutilized.
- Load balancing is the process of distributing workloads (requests, traffic, or VM tasks) evenly across multiple servers or virtual machines.

Why it's needed:

- To improve performance and response time.
- To ensure high availability (if one server fails, another can take over).
- To use resources efficiently (CPU, memory, storage).

Example:

- In a cloud data center, if one server running VMs reaches 90% CPU usage, the load balancer may migrate some VMs to another server at 40% CPU usage.
- Tools: VMware vMotion, Microsoft System Center, HAProxy, AWS Elastic Load Balancer.

Google Cloud Load Balancing

When you use Google Cloud Load Balancing, you get one global IP address for your app or website. No matter where the user is located, they always connect through this single IP.

When a user connects, Google automatically sends them to the nearest server. This ensures faster response times and a better experience for the user.

If one server becomes busy or fails, the load balancer redirects the traffic to another healthy server. This way, the application always stays available.

Google Cloud Load Balancing also supports autoscaling. It can add more servers automatically when the traffic increases and remove extra servers when the traffic decreases.

It works for different types of traffic, such as web applications (HTTP/HTTPS), network traffic (TCP/UDP), or even internal applications inside a private cloud.

Example:

Example:

- A website is hosted on Google Cloud in India, USA, and Europe.
- Users from India connect to India servers, US users connect to US servers.
- If India's server fails, traffic is sent to Singapore or another nearby region.

Layered structure of virtualization in cloud computing.

Layered Structure of Virtualization in Cloud Computing

- Virtualization in cloud computing is built in a layered structure, where each layer has a specific role. These layers work together to create and manage virtual machines (VMs) and deliver cloud services.

1. Hardware Layer

- This is the physical layer (CPU, memory, storage, network).
- It provides the actual resources that will be shared among virtual machines.

2. Hypervisor Layer

- Also called Virtual Machine Monitor (VMM).
- It sits on top of hardware and divides resources into virtual machines.
- Two types: Type 1 (bare-metal) and Type 2 (hosted) hypervisors.

3. Virtual Machine Layer

- Virtual machines (VMs) are created by the hypervisor.
- Each VM has its own operating system and applications, working independently like a real computer.

4. Virtual Infrastructure Management Layer

- Provides management tools for admins to control VMs.
- Handles resource allocation, monitoring, load balancing, migration, and scaling.
- Examples: VMware vCenter, OpenStack.

5. Application / Cloud Services Layer

- This is the top layer where users interact.
- Provides cloud services (IaaS, PaaS, SaaS) using the underlying virtualized infrastructure.
- Examples: Google Cloud, AWS, Microsoft Azure.

Google Cloud Applications (e.g. Google Analytics, Translate).

1. Google Analytics

- Google Analytics is a web analytics service that helps track and report website traffic.
- It provides information about user behavior, such as the number of visitors, their location, devices used, and pages they visit.
- Businesses use it to improve marketing strategies, user experience, and website performance.
- Example: An e-commerce site can use Google Analytics to see which products are most visited.

2. Google Translate

- Google Translate is a machine translation service that supports over 100 languages.
- It uses neural machine translation (NMT), a type of AI, to provide more accurate translations.
- It can translate text, speech, images, and even websites in real time.
- Example: A traveler can instantly translate a restaurant menu using the Google Translate mobile app.

Type I and Type II Hypervisors with examples:

Feature	Type I Hypervisor	Type II Hypervisor
Layer	Runs directly on hardware	Runs on top of host operating system
Performance	High	Lower (due to extra OS layer)
Security	More secure	Less secure
Use Case	Servers, cloud computing, data centers	Personal computers, testing environments
Installation	Complex	Easy (just like installing a normal application)
Examples	ESXi, Hyper-V Server, Xen, KVM	VirtualBox, VMware Workstation, Parallels, Fusion

Amazon EC2 (Elastic Compute Cloud)

Amazon EC2 is a cloud service that provides virtual machines (called EC2 instances) to run applications. Instead of buying physical servers, companies can rent these virtual servers on demand. EC2 allows you to choose the amount of CPU, memory, storage, and operating system according to your need. It is widely used for hosting websites, running applications, data processing, and software testing. The main advantage of EC2 is flexibility—you can start or stop servers anytime and pay only for the time you use. For example, a company can quickly launch an EC2 instance to host a web application during high traffic hours and shut it down later to save cost.

Amazon S3 (Simple Storage Service)

Amazon S3 is an online storage service where you can store and retrieve any amount of data at any time. It works like a large storage bucket in the cloud. Users and companies use S3 to store images, videos, documents, backups, logs, and even large datasets. One of its biggest advantages is durability and reliability; AWS automatically stores multiple copies of your data so it is safe

from loss. S3 is commonly used for website hosting, storing application data, keeping backup files, and delivering content to users. For example, a mobile app can store all its user-uploaded photos in S3 because it is cheap, secure, and easily accessible.

Amazon SimpleDB

Amazon SimpleDB is a NoSQL database service used for storing and querying small structured data. It is simpler than traditional relational databases and does not require complex setup or management. SimpleDB automatically manages indexing, data organization, and performance. It is useful when you need a lightweight database with fast reads and writes but do not require complex SQL queries. For example, an e-commerce website can use SimpleDB to store product details such as name, price, and category because the data is simple and needs quick access. Although newer services like DynamoDB are more popular now, SimpleDB is still useful for simple structured storage.

What are non-relational data models in the cloud with MongoDB as an example. How does the CAP theorem apply here?

Non-Relational Data Models in the Cloud

Non-relational data models, also known as NoSQL databases, are designed to store large amounts of data that may not fit neatly into rows and tables like traditional relational databases. They are flexible, can handle unstructured or semi-structured data, and are built to scale easily across many servers in the cloud. Instead of using tables, NoSQL databases may use key-value pairs, documents, graphs, or wide-column structures. These databases are often used in modern cloud applications where data grows very fast, such as social media apps, e-commerce platforms, and real-time analytics systems.

MongoDB as an Example

MongoDB is one of the most popular NoSQL databases, especially in cloud environments. It stores data in JSON-like documents (called BSON), which makes data easy to read and flexible. Each document can have different fields, and you don’t need to define a fixed schema like in relational databases.

In the cloud, MongoDB easily scales horizontally—meaning you can add more servers to handle more data or traffic. It supports features like automatic sharding (splitting data across machines), replication (copying data for reliability), and easy backup. For example, an online shopping website can store product details, customer info, and reviews in MongoDB because the data structure changes often and grows quickly.

How the CAP Theorem Applies

The **CAP Theorem** states that in a distributed database system, you can only guarantee **two** of the following three properties at the same time:

1. **C – Consistency**
   Every user sees the same data at the same time.
2. **A – Availability**
   The system continues to work and respond to requests even if some servers fail.
3. **P – Partition Tolerance**
   The system keeps working even if communication between servers is broken or delayed.

A cloud database **must** handle network partitions (P), because cloud systems involve many servers communicating over networks.
So the real choice is between **Consistency (C)** and **Availability (A)**.

CAP Theorem in MongoDB

MongoDB allows different configurations, but generally it prefers Availability (A) and Partition Tolerance (P) over strict Consistency.
This means:

- If a network failure happens, MongoDB can still accept requests by switching to another replica.
- Some users may temporarily see slightly older data (called *eventual consistency*).
- MongoDB tries to keep working even if part of the cluster is unreachable.

However, MongoDB also gives users control. You can choose strong consistency by using the primary node for reads, or choose higher availability by allowing reads from secondary replicas.

Example Scenario:
If one server in a MongoDB cluster goes down, the system will still respond using other servers, ensuring high availability. But for a moment, the latest update might not appear to all users immediately. This is the CAP trade-off.

Auditing and Compliance Requirements in a Cloud Environment

Auditing and compliance are important in cloud computing because organizations must make sure their data is safe, properly handled, and follows laws or industry rules. In a cloud environment, auditing means checking how data is stored, who is accessing it, and whether security controls are working correctly. Compliance refers to meeting standards such as GDPR, HIPAA, ISO 27001, or government regulations. Since cloud resources are shared and managed by external providers, organizations must ensure that the cloud provider follows proper security practices, maintains data privacy, and provides

logs and reports for verification. Regular audits help detect mistakes, security risks, or unauthorized access, and they ensure that the cloud environment follows all required policies.

How Third-Party Providers Ensure Transparency

Third-party cloud providers can ensure transparency by giving customers clear information about their security controls, data-handling processes, and compliance certifications. They can provide audit reports (like SOC 1, SOC 2, ISO certifications) that prove their systems meet industry standards. Providers should also offer detailed logging and monitoring tools so users can see who accessed their data and when. Transparency is further improved when cloud providers share service-level agreements (SLAs), incident reports, and updates about any changes in the system. Many providers allow independent third-party audits to check whether their security practices are reliable. By openly sharing these results and providing dashboards for continuous monitoring, cloud providers build trust and help organizations maintain compliance.

Role of SOA in Modern Cloud Applications

Service-Oriented Architecture (SOA) plays an important role in modern cloud applications by breaking large systems into small, independent services that can communicate with each other over a network. Each service performs a specific function and can be reused in different applications. In cloud environments, where scalability and flexibility are essential, SOA makes it easier to build applications that can grow or shrink based on user demand. It also allows developers to update or replace individual services without affecting the whole system. Because cloud platforms often integrate services from different providers, SOA helps ensure that these services can work together smoothly using standard interfaces. This makes cloud applications more modular, cost-effective, and easier to maintain.

How SOA Protocols Improve Workflow Coordination in Distributed Systems

SOA uses standard protocols such as SOAP, REST, XML, JSON, and WSDL to enable communication between distributed services. These protocols ensure that services built on different platforms and programming languages can still interact smoothly. By using well-defined message formats and service contracts, SOA protocols make workflows predictable and reliable. For example, SOAP uses structured XML messaging that ensures secure and consistent communication, while REST provides lightweight and fast interactions over HTTP. In distributed systems, workflows often depend on multiple services working together—such as authentication, payment, and notification. SOA protocols help coordinate these actions by providing clear communication rules, error-handling mechanisms, and support for synchronous or asynchronous messaging. This results in better coordination, faster integration, and more reliable execution of complex workflows

Shift in Paradigm from Traditional IT to Cloud

The move from traditional IT to cloud computing represents a major shift in how organizations build and use technology. In traditional IT, companies must buy physical servers, maintain data centers, install software manually, and hire technical staff for maintenance. This requires high cost, more time, and fixed capacity. In contrast, cloud computing provides on-demand access, meaning organizations can use resources whenever needed and pay only for what they use. Cloud services are also scalable, so companies can increase or decrease computing power instantly. Instead of owning hardware, organizations now rent IT resources, reducing capital expenditure. The cloud also improves flexibility because users can access services from anywhere through the internet. This shift changes the focus from managing infrastructure to using ready-made services, leading to faster innovation, lower cost, and easier maintenance.

Possible security issues during migration

Migration opens new attack surfaces if not designed carefully. Key risks are:

- Data interception (eavesdropping): memory pages and device state transferred over the network may contain sensitive data; unencrypted transfer risks leakage.
- Man-in-the-middle or injection attacks: if migration channels are not authenticated or integrity-protected, an attacker could alter state in transit or inject malicious state.
- Compromise of migration module / hypervisor: vulnerabilities in the migration code or VMM could let attackers gain control of the hypervisor and all hosted VMs.
- Roll-back or replay attacks: stale or manipulated state sent to the target can corrupt VM execution or leak info.
- Side-channel leaks: migration activity patterns (timing, volume) may leak information about workload.

Live migration of virtual machines

Live migration of virtual machines means moving a running virtual machine (VM) from one physical server to another without shutting it down.
The VM continues to run, and users experience either zero or very minimal downtime.

Need for live migration of virtual machines and the design process

Live migration lets a running VM move from one physical host to another without (or with minimal) downtime. Need arises for several reasons: avoiding planned maintenance, balancing load, consolidating for energy savings, disaster avoidance (move VMs off a failing host), and enabling proactive fault management. In clouds and data centers where services must stay up, live migration supports high availability and flexible resource management.

Typical design process (high level):

1. **Pre-migration checks:** ensure target host has resources (CPU, RAM, storage, network), compatible hypervisor and policy checks.
2. **State transfer (memory & device state):** iterative pre-copy phases send the VM's memory pages to the target while the VM still runs; pages dirtied during transfer are re-sent in subsequent rounds.
3. **I/O and disk handling:** either use shared storage (so disk images are available on both hosts) or block-device replication/synchronization.
4. **Stop-and-copy (final switchover):** briefly pause the VM, send final dirty pages and device state, update network routing, then resume the VM on the target. The goal is to minimize the paused time.
5. **Post-migration cleanup:** release resources on the source and ensure monitoring/networking reflect the new location. Research surveys and design papers categorize these steps and variants (pre-copy, post-copy, hybrid) and discuss tradeoffs of downtime vs network cost.

Azure Content Delivery Network (CDN)

Azure Content Delivery Network (CDN) is a cloud service provided by Microsoft Azure that helps deliver web content to users faster and more efficiently. It works by storing copies of files—such as images, videos, scripts, CSS, and downloadable content—in multiple servers located around the world, called edge locations. When a user accesses a website or application, the content is delivered from the nearest edge server, reducing delay and improving loading speed.

Azure CDN reduces network congestion, boosts website performance, and handles high traffic loads easily. It supports dynamic site acceleration, HTTPS for secure delivery, caching, and integration with Azure services like Blob Storage, Web Apps, and Media Services. It is commonly used for streaming videos, delivering large files, improving mobile app performance, and providing a fast user experience across global regions.

Azure AppFabric

Azure AppFabric was a set of cloud-based middleware services provided by Microsoft to help developers build, connect, and manage scalable applications on the Azure platform. It acted as a bridge between on-premises systems and cloud services, making it easier to integrate distributed applications.

AppFabric mainly offered two key services:

- AppFabric Service Bus – Enabled reliable communication between different applications, even if they were behind firewalls or in different networks. It supported messaging patterns such as queues, publish–subscribe, and relays.
- AppFabric Access Control Service (ACS) – Provided identity and access management, allowing applications to use federated authentication through identity providers like Active Directory, Google, and others.

Azure AppFabric simplified building hybrid applications, supported secure communication, and reduced the complexity of distributed system integration. Although later replaced by newer Azure services (like Azure Service Bus and Azure Active Directory), AppFabric played an important role in the early evolution of Microsoft's cloud platform.

Cloud Auditing Outsourcing

Cloud auditing outsourcing means hiring an external, independent third-party to check and verify the security, performance, privacy, and compliance of a cloud service or cloud provider. Instead of the organization conducting the audit on its own, it depends on certified auditors who have expertise in cloud standards and best practices.

The outsourced auditor reviews areas such as data protection, access controls, encryption, service reliability, legal compliance (like GDPR or HIPAA), and the provider's internal policies. They also inspect logs, documentation, and controls to ensure that the cloud environment meets required regulations.

Organizations outsource cloud auditing because it is cost-effective, provides unbiased evaluation, and helps identify risks that internal teams may overlook. This also builds trust between cloud providers and customers, since independent audits offer transparent, verified proof of security and compliance.